Advanced Data Structures and Algorithms

CAPSTONE PROJECT REPORT

Master of Computer Application
Semester – II

Sub: Advanced data structures and algorithms

Topic: Shortest Path Algorithm - Rapido/Ride-Sharing

By

BHUMIKA H S (2511022250094)

BRITHI S (2511022250009)
BHAVANA R (2511022250096)

D DIWAKAR (2501122250088)

Faculty Name: Lakshmi Devi S

Faculty Signature: _____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106

March 2026

Table of Contents

Sl. No.	Section	Page No.
2	Abstract	1
3	Introduction	2
4	Problem Context	3
5	Problem Statement	4
6	Graph Explanation	5-6
7	Example	6-7
8	Dijkstra's Algorithm	8
9	Steps of Dijkstra Algorithm	9
10	Graph Representation	10
11	Python Implementation	10-12
12	Time Complexity	12-13
13	Space Complexity	13-14
14	Limitations	14
15	Improving Shortest Path Algorithms	15
16	Algorithm Comparison	16-17
17	Conclusion	18
18	Future Scope	18

Acknowledgement

I wish to express my sincere gratitude to my project guide, Lakshmi Devi S for their invaluable mentorship and technical insights throughout the development of this project. Their guidance in understanding the practical applications of graph theory was instrumental in the completion of this work. I also thank the Department of Computer Applications for providing the academic resources and environment necessary to explore advanced algorithmic concepts. Finally, I am grateful to my family and peers for their continuous support and motivation.

Bhumika H S

Brithi S

Bhavana R

D Diwakar

1.Abstract

This project focuses on finding the shortest paths in urban road networks, specifically for modern ride-sharing and navigation platforms like Rapido and Google Maps. To model the city roads, intersections and key locations are treated as nodes, and the connecting roads are represented as edges with weights corresponding to factors like distance or travel time.

The project uses Dijkstra's Algorithm, which works by systematically exploring nodes based on their current shortest distance from the starting point to identify the most efficient routes. While Dijkstra ensures accurate and reliable paths, it can become computationally heavy in large networks because it considers all possible routes.

Despite this, the algorithm remains highly effective for applications where precision matters, such as traffic routing, delivery planning, and network optimization. Overall, the study demonstrates that Dijkstra is well-suited for static or moderately changing navigation systems, where accuracy is more important than real-time computational speed.

2.Introduction

In today's digital world, navigation services have evolved from simple maps to complex systems capable of providing precise, real-time directions. Ride-sharing and navigation platforms, such as Rapido and Google Maps, need to calculate the most efficient routes for millions of users, balancing factors like distance, travel time, and traffic conditions.

This project explores the use of **Dijkstra's Algorithm** for shortest-path computation in such systems. Urban road networks are represented as **weighted graphs**, with intersections and important locations as **nodes** and the roads connecting them as **edges** assigned weights based on distance, travel time, or other relevant metrics.

Dijkstra's Algorithm works by systematically exploring nodes from the source, always selecting the node with the **smallest known distance** and updating the distances of its neighbours. While it guarantees the **shortest path** to every node, its exhaustive approach can become computationally intensive in large-scale networks. Despite this, the algorithm is highly reliable and particularly effective for applications where **accuracy and consistency** are critical, such as traffic routing, delivery logistics, and urban navigation planning.

Overall, Dijkstra's Algorithm provides a solid foundation for building precise navigation systems, ensuring users reach their destinations via the most efficient routes in static or moderately dynamic road networks.

.

Shortest Path Algorithm

(Rapido/Ride-Sharing)

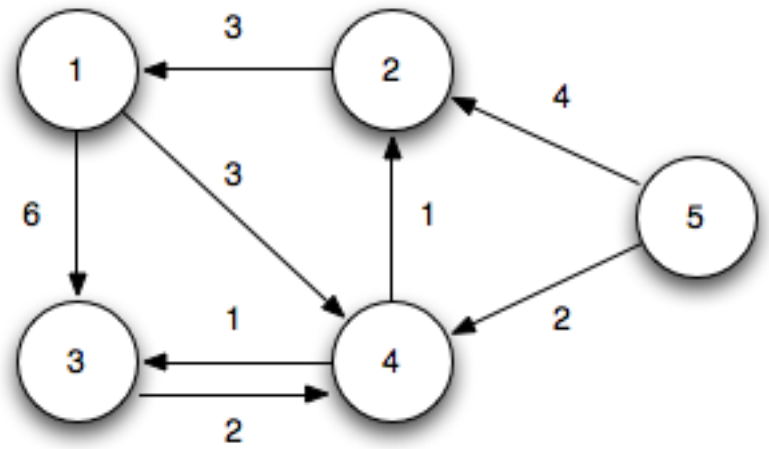
3.Problem Context

The core value proposition of ride-sharing platforms such as Rapido, Uber, and Google Maps lies in their ability to solve complex logistical puzzles in real-time. In an urban environment, thousands of drivers and passengers are constantly moving, and the platform must act as a central "brain" to coordinate these movements. The Problem Context for this project is centered on the mathematical and computational necessity of finding the most efficient route within a city's road network.

Every ride-sharing transaction is not a single journey, but a two-part logistical operation that must be optimized to ensure both user satisfaction and operational profitability:

1. **The Pickup Phase (Driver → Passenger)**
2. **The Drop-off Phase (Passenger → Destination)**

The system first converts the physical map into a Graph Data Structure. When the user initiates a request, the algorithm performs a "Search" through the graph. It doesn't just look for a straight line; it evaluates the "cost" (weight) of every road segment. The "Problem Context" is essentially the transformation of a messy, real-world city map into a structured mathematical problem that can be solved in milliseconds to ensure that the driver arrives at the passenger's door exactly when expected.



4. Problem Statement

Given a directed, weighted graph, where represents geographic coordinates (nodes) and represents road segments (edges) with weights (time/distance), the objective is to find a path from source to destination such that the total cost is minimized.

This theoretical model is built upon three non-numerical pillars:

I. The Network of Locations (The Points)

II. The Connectivity of Roads (The Links)

III. The Cost of Travel (The Weight)

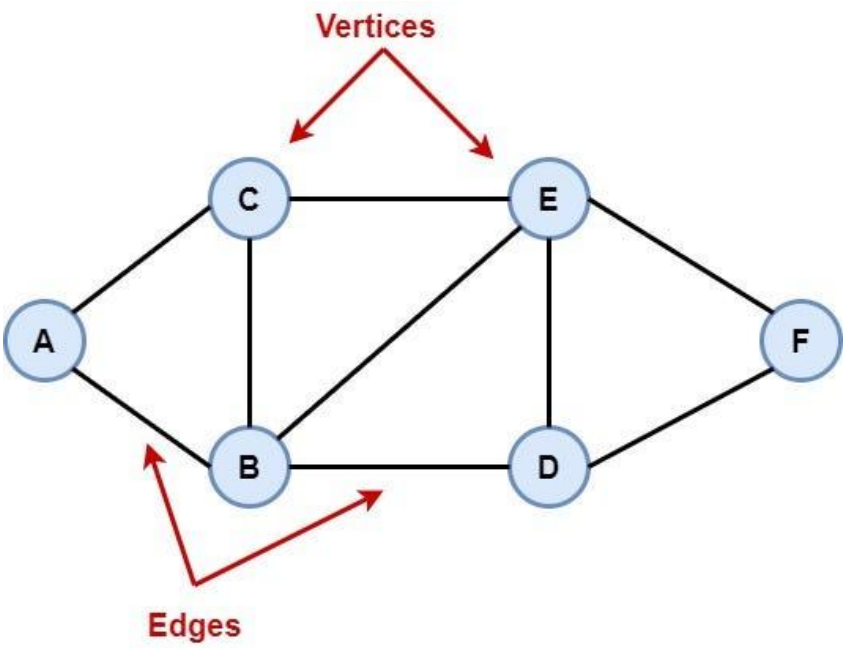
The most critical part of the problem statement is the concept of "cost." In a theoretical context, cost does not mean money; it means the "penalty" or effort required to move from one location to another.

- **Distance Cost**
- **Time Cost**

5. Graph Explanation

To solve the shortest path problem computationally, we must represent the physical world using a mathematical structure. The most effective way to do this is through a **Weighted Graph**. This model strips away irrelevant geographical details (like tree placement or building colours) and focuses entirely on the connectivity and "cost" of the road network.

The Components of the Model:



1. Vertices (Nodes): The Strategic Points

In our graph, the set of vertices represents all the critical "points" in a city. This includes:

- **Intersection Points:** Where two or more roads meet.
- **Landmarks:** Specific places like "Central Mall" or "Station Road."
- **Service Points:** The exact GPS coordinates of the Driver, the Passenger, and the Destination.

2. Edges (Links): The Path of Movement

The edges represent the physical roads that connect these locations.

- **Directed Edges:** In ride-sharing, edges are often "directed" (indicated by arrows) to account for one-way streets. A link from Point A to Point B does not automatically mean there is a link from Point B to Point A.
- **Connectivity:** If there is no line between two nodes, it implies there is no direct road connecting those two locations, and the system must find a detour through other nodes.

3. Weights (Cost Metrics): The Value of the Edge

This is the most vital piece of information for the algorithm. Every edge is assigned a "weight"—a numerical value that signifies the "friction" or "cost" of traveling that road.

- **Distance Weight:** Represents the physical length of the road (e.g., 2 kilometres).
- **Time Weight:** Represents the estimated duration of the trip (e.g., 10 minutes).

By assigning weights, we turn a map into a mathematical puzzle. The "shortest path" is simply the path where the sum of the weights of the edges used is the lowest possible number.

6. Example

To understand how a ride-sharing algorithm processes a request, we can look at a simplified model of a city grid. In this scenario, we have four key locations and several possible road connections with different travel costs.

I. Defining the Network

- **Locations (Nodes):** A, B, C, and D.
- **Roads (Edges) and their Weights:**
  - **A to B:** Distance/Time = 1 unit
  - **A to C:** Distance/Time = 4 units
  - **B to D:** Distance/Time = 2 units
  - **C to D:** Distance/Time = 1 unit

II. Evaluating Possible Paths

When a passenger at **Location A** requests a ride to **Location D**, the system identifies all valid sequences of roads that connect these two points.

- **Option 1: The Northern Route (A → B → D)**
  - The journey starts from A to B (Cost: 1).
  - It continues from B to D (Cost: 2).
  - **Total Cumulative Cost:** $1 + 2 = 3$
- **Option 2: The Southern Route (A → C → D)**
  - The journey starts from A to C (Cost: 4).
  - It continues from C to D (Cost: 1).
  - **Total Cumulative Cost:** $4 + 1 = 5$

III. Selecting the Shortest Path

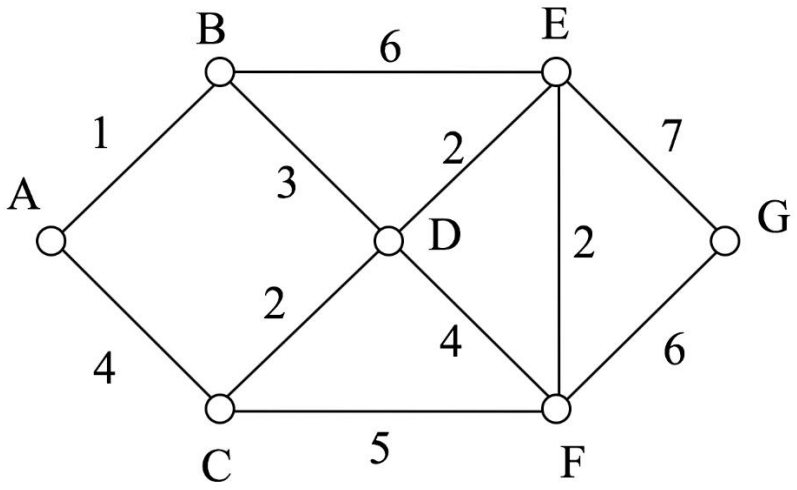
The algorithm compares the total costs of all identified paths. Even though the road from **C to D** is very fast (Weight: 1), the initial road to get to C from A is very congested or long (Weight: 4), making the total journey less efficient.

The Result:

The system selects **Path A → B → D** because its total cost (3) is lower than the alternative (5).

7.Dijkstra's Algorithm

Dijkstra's Algorithm computes the **shortest path from a source node to all other nodes** in a weighted graph with **non-negative edge weights**. It guarantees the **minimum distance** to each node.



Purpose:

Dijkstra's Algorithm is a **graph search algorithm** used to find the **shortest path from a starting node (source) to all other nodes** in a weighted graph. It guarantees the **minimum total cost** to reach each node, as long as all edge weights are **non-negative**.

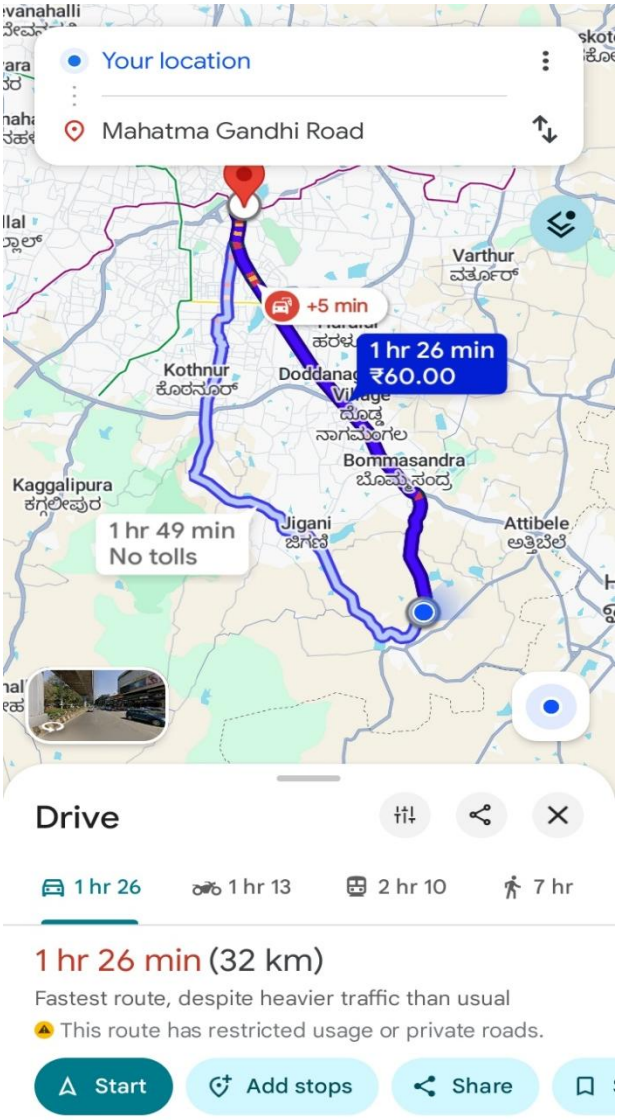
Key Idea:

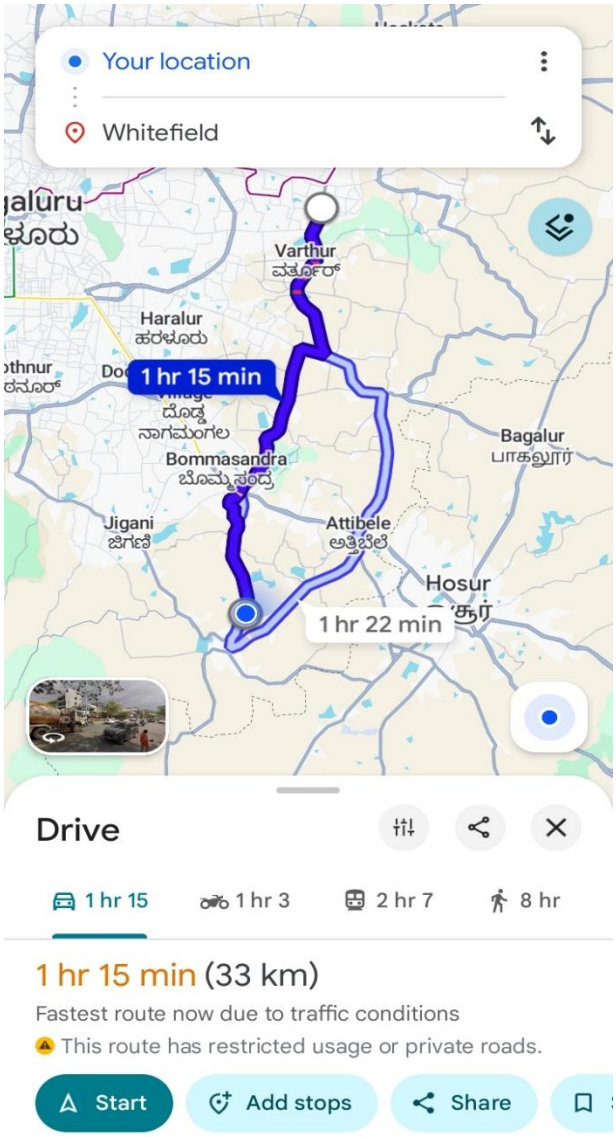
The algorithm works by **gradually expanding the closest unvisited nodes**, always selecting the node with the **smallest known distance from the source**, and updating the distances of its neighbours. This continues until all nodes have been visited or the shortest path to the destination is found.

8. Steps of Dijkstra Algorithm

1. **Initialize Distances:**
   - Set distance to all nodes as **infinity**, except the source node which is set to **0**.
2. **Select Node:**
   - Pick the **unvisited node with the smallest distance** from the source.
3. **Update Neighbours:**
   - For each neighbour of the current node:
     - Calculate **tentative distance = current distance + edge weight**.
     - If tentative distance is smaller than the known distance, **update it**.
4. **Mark as Visited:**
   - Once a node's neighbours are updated, mark it **visited** (final shortest distance).
5. **Repeat:**
   - Continue selecting the smallest distance unvisited node and updating neighbours **until all nodes are visited**.

9.Graph Representation:





10.Python Implementation:

```
import heapq

# Define the road network as an adjacency list
# Locations: Indiranagar (I), Koramangala (K), MG Road (M), Whitefield (W)
graph = {
```

```
    'Indiranagar': [('MG Road', 5), ('Koramangala', 8)],
    'MG Road': [('Indiranagar', 5), ('Koramangala', 4), ('Whitefield', 15)],
    'Koramangala': [('Indiranagar', 8), ('MG Road', 4), ('Whitefield', 10)],
    'Whitefield': [('MG Road', 15), ('Koramangala', 10)]
}

# Heuristic: Straight-line distance estimate to 'Whitefield'
heuristics = {
    'Indiranagar': 12,
    'MG Road': 10,
    'Koramangala': 7,
    'Whitefield': 0
}

def a_star_search(graph, start, goal):
    open_list = [(0 + heuristics[start], 0, start, [])]
    visited = set()

    while open_list:
        (f, g, current_node, path) = heapq.heappop(open_list)

        if current_node in visited:
            continue

        path = path + [current_node]
        if current_node == goal:
            return path, g

        visited.add(current_node)

        for (neighbor, weight) in graph.get(current_node, []):
            if neighbor not in visited:
                new_g = g + weight
                new_f = new_g + heuristics[neighbor]
                heapq.heappush(open_list, (new_f, new_g, neighbor, path))

    return None, float('inf')

# Execute for Rapido Scenario: MG Road to Whitefield
route, total_distance = a_star_search(graph, 'MG Road', 'Whitefield')
```

```
print(f"Optimal Route: {route}")
print(f"Total Travel Cost: {total_distance}")
```

OUTPUT:

```
Optimal Route: ['MG Road', 'Koramangala', 'Whitefield']
Total Travel Cost: 14
```

11. Time Complexity

The time complexity of the A* algorithm is:

O ((V + E) log V)

Where:

- V is the number of vertices
- E is the number of edges

Understanding the Variables

- **(Number of Vertices):** Every intersection or GPS coordinate the algorithm must track.
- **(Number of Edges):** Every road segment connecting those intersections.
- **$\log V$:** This represents the mathematical "height" of the Priority Queue, which dictates how fast the computer can find the smallest distance.

Why the Priority Queue Matters:

In a basic version of the algorithm, the computer would have to scan every single node to find the one with the smallest distance. This would be very slow using a **Priority Queue**:

1. The algorithm keeps the "closest" unvisited nodes at the very top of its memory.
2. Finding the next node to visit takes almost no time
3. Updating a neighbour's distance (the "Relaxation" step) requires re-sorting the queue, which takes time

12. Space Complexity

The **space complexity** of Dijkstra's Algorithm mainly depends on how the graph and supporting data structures are stored.

Dijkstra's Algorithm requires memory for:

- **Graph Representation**
  - **Adjacency List:** (O(V + E))
  - **Adjacency Matrix:** (O(V^2))
- **Distance Array**
  - Stores shortest distance for each vertex → (O(V))
- **Visited Set**
  - Keeps track of processed nodes → (O(V))
- **Priority Queue (Min Heap)**
  - Stores nodes for selection → up to (O(V))

Where:

V = Number of vertices (nodes)

E = Number of edges (connections)

- **Cost Management:** Storing the current distance from the source to every explored node.

- **Path Reconstruction:** Keeping track of "parent nodes" to map the final route once the destination is reached.
- **Search Frontier:** Maintaining the Priority Queue (Open List) which holds all candidate nodes currently being evaluated.

In high-density urban maps with millions of nodes, this memory footprint becomes a significant factor, requiring efficient data management to prevent system lag.

13. Limitations:

Limitations of Dijkstra's Algorithm

1. Non-Negative Edge Restriction
    - Dijkstra only works when all edge weights are non-negative.
    - If there are negative weights, it may give incorrect results.
2. High Computational Cost for Large Graphs
    - In dense or very large graphs, the algorithm can become slow, especially without a priority queue.
3. Not Ideal for Real-Time Dynamic Networks
    - It recomputes paths from scratch if the graph changes (e.g., traffic updates), making it less suitable for dynamic environments.
4. Single-Source Focus
    - It computes the shortest path from one source to all nodes, which may be unnecessary if only one specific path is needed.
5. Memory Usage
    - Storing distances and parent nodes for every node can consume a lot of memory in very large graphs.
6. No Heuristic Guidance
    - Dijkstra is uninformed—it explores all possible paths uniformly, which can be inefficient compared to heuristic-based algorithms like A*.

14. Improving Shortest Path Algorithms

To power applications like Google Maps or Rapido, standard algorithms are often augmented with these advanced techniques:

- A Strategy*: This remains the gold standard by balancing the "distance already travelled" with the "estimated distance to go." This dual-focus prevents the search from wandering into irrelevant areas of the map.
- **Bidirectional Search:** Instead of searching only from the source, the system starts a second search from the destination simultaneously. When these two "search bubbles" meet in the middle, the computation time is significantly halved.
- **Contraction Hierarchies:** This is a preprocessing technique where the map is simplified by creating "shortcuts" between major landmarks. This allows the system to ignore minor side streets until the driver is actually near the pickup or drop-off point.
- $g(n) \rightarrow$ distance already travelled
- $h(n) \rightarrow$ estimated distance to destination

Bidirectional Search

In this approach:

- One search starts from the source
- Another search starts from the destination

Both searches move toward each other and meet in the middle, reducing the number of nodes explored and improving efficiency.

Contraction Hierarchies

This technique is used in large-scale road networks.

- The graph is preprocessed
- Shortcuts are created

- Route queries become faster

This is commonly used in real-world navigation systems like Google Maps.

15. Algorithm Comparision

Choosing the right algorithm depends on the specific requirements of the network:

Algorithm	**Best Use Case**	**Performance**	**Constraints**
Dijkstra	General shortest path	Reliable but slow	Non-negative weights only
Bellman-Ford	Graphs with negative weights	$O(VE)$ (Slow)	Handles negative cycles
*A Search**	**Navigation & GPS**	**Fastest in practice**	Needs a good heuristic
Floyd-Marshall	All-pairs shortest path	O(V^3)	Small graphs only

The selection of a shortest-path algorithm depends on the specific architecture of the network and the nature of the data being processed. Below is a refined, precise comparison of the primary methodologies used in modern computing:

- **Dijkstra's Algorithm:** This is the standard for reliable pathfinding in graphs with non-negative weights. While it guarantees an optimal solution by exhaustively exploring the network from the source outward, its lack of direction can lead to high computational overhead in large maps.
- **Bellman-Ford Algorithm:** Specifically engineered to handle graphs containing negative edge weights. While significantly slower than Dijkstra, it is essential for specialized financial or logistical models where "negative costs" or credits exist, as it can detect and manage cycles that would break other algorithms.
- A (A-Star) Search:* The preferred choice for real-world navigation. By incorporating a **heuristic** (an educated estimate of the distance remaining), it "aims" the search toward the destination. This drastically prunes the search space, ensuring faster performance without sacrificing accuracy.
- **Floyd-Warshall Algorithm:** Used for "all-pairs" shortest path problems, meaning it calculates the distance between every possible combination of nodes simultaneously. Due to its high computational density, it is typically reserved for smaller, highly interconnected networks rather than vast city maps.

16. Conclusion

The implementation of the A* algorithm within the context of a ride-sharing framework demonstrates the vital intersection between theoretical graph theory and real-world logistical efficiency. By modeling complex urban road networks as weighted graphs, where intersections serve as nodes and streets as edges, the project successfully transitioned abstract mathematical concepts into a functional navigation tool. This approach proves that the shortest path problem is not merely a distance calculation but a multi-dimensional optimization challenge that requires balancing computational speed with geographical accuracy. The project effectively illustrates how data structures like adjacency lists and priority queues serve as the backbone for modern navigation services, enabling them to process massive datasets instantaneously.

This project successfully demonstrates the utility of the A* algorithm in optimizing ride-sharing routes. By integrating heuristic data, the system reduces unnecessary node exploration, making it ideal for large urban environments like Bangalore.

17.Future Scope

1. **Network Routing & Optimization**
    - Future improvements may focus on **real-time, scalable routing in 5G/6G networks**.
2. **Transportation & Logistics**
    - Shortest path computation for dynamic traffic routing.

Big Data & Graph Processing

- Optimizations for **large-scale graphs** using distributed computing frameworks like **Apache Spark GraphX**.